

Distinct Project

DOCUMENTATION

1. Project Overview (Distinct):

- The goal of this project is to create a function that takes Array A of N integer and returns the total count of unique (distinct) values present in the array.

- **Input (N):** 0 to 100,000
- **Example:** For A = [2, 1, 2, 3, 3], the result is 3 (the distinct values are 1, 2 and 3).

2. First Algorithm (Non-Recursive):

Psuedo Code:

Algorithm count_distinct(array, n)

 for i ← 0 to n-1 do

 for j ← i+1 to n-1 do

 if array[j] < array[i] then

 SWAP(array[i], array[j])

count ← 0

for i ← 0 to n-1 do

 if i == 0 OR array[i] ≠ array[i-1] then

 count ← count + 1

return count

Implementation:

```
int count_distinct(int array[], int n) {
    int count = 0;
    for (int i = 0; i < n; i++) {
        for (int j = i + 1; j < n; j++) {
            int temp = 0;
            if (array[j] < array[i]) {
                temp = array[j];
                array[j] = array[i];
                array[i] = temp;
            }
        }
    }

    for (int i = 0; i < n; i++) {
        if (i == 0 || array[i] != array[i - 1]) {
            count++;
        }
    }

    return count;
}
```

Analysis & Complexity

A. Analysis:

- **Phase 1 – Sorting:** The function uses a nested loop to sort the array in ascending order. The outer loop runs **n** times and the inner loop runs **(n-i-1)** times, resulting in: $\frac{n(n-1)}{2}$
- **Phase 2 – Counting:** A single loop runs on the sorted array once, comparing each element to its previous one to count unique values, costing **n** operations.

B. Total Operations: $\frac{n(n-1)}{2} + n = \frac{n^2+n}{2}$.

C. Total Complexity: $O(n^2)$, this means doubling the array size, quadruples the running time.

D. Space Complexity: $O(1)$

Analysis & Complexity Steps:

1- Input loop:-

$$T(n) = n \quad \text{Time Complexity} = O(n)$$

2- Sorting nested loop:- "Selection Sort"

Outer loop:- runs n times

Inner loop:-

if $i=0$ runs $(n-1)$

if $i=1$ runs $(n-2)$

if $i=2$ runs $(n-3)$

if $i=n-1$ runs 0

inner loop runs $(n-1-i)$

$$T(n) = (n-1) + (n-2) + (n-3) + \dots + 1$$

Total Complexity:-

$$\text{Note: } \sum_{i=1}^n i = \frac{n(n+1)}{2}$$

$$T(n) = \sum_{i=0}^{n-1} (n-1-i)$$

$$T(n) = \frac{(n-1) \cdot n}{2}$$

$$\text{Total Time Complexity} = O(n^2)$$

3- The count

$$T(n) = n$$

Time complexity :- $O(n)$

Space complexity :- $O(1)$

4- over all :-

Time complexity :- $O(n) + O(n^2) + O(n) = O(n^2)$

Space complexity :- $O(n)$

5- ~~A~~ cases :-

best case :- $O(n^2)$

worst case :- $O(n^2)$

average case :- $O(n^2)$

3. Second Algorithm (Recursive):

Pseudo Code:

Algorithm Count_Distinct(A, size, index)

```
    if index = size then
        return 0
    y ← Exists (A, index, index - 1)
    if y = true then
        return Count_Distinct(A, size, index + 1)
    return 1 + Count_Distinct(A, size, index + 1)
```

Algorithm Exists (A, index, search)

```
    if search < 0 then return false
    if A[index] = A[search] then return true
    return Exists (A, index, search - 1)
```

Implementation:

```
bool exists(int arr[], int index, int search) {
    if (search < 0) return false;
    if (arr[index] == arr[search]) return true;
    return exists(arr, index, search - 1);
}

int count_distinct(int arr[], int size, int index) {
    if (index == size) return 0;
    if (exists(arr, index, index - 1))
        return countDistinct(arr, size, index + 1);
    return 1 + countDistinct(arr, size, index + 1);
}
```

Analysis & Complexity:

A. Complexity of Exist Function:

- **Recurrence Relation:** $T(s) = T(s-1) + 1$. where s is the index of element to be searched.
- **Base Case:** $T(-1) = 1$. (When $\text{search} < 0$), this means there are no elements to search.
- **Result:** $O(n)$

B. Complexity of CountDistinct Function:

- **Recurrence Relation:** $T(i) = i + 1 + O(i)$, where i is the index of the element.
- **Logic:** For each element, the Exists function is called, costing $O(n)$ in the worst case.
- **Base Case:** $T(i)=1$. (When $\text{size} = \text{index}$).
- **Expansion:** $T(n) = \frac{n(n-1)}{2}$
- **Result:** $O(n^2)$

C. Complexity of Total Program:

- The program consists of `insert_arr` ($O(n)$) and `count_distinct` ($O(n^2)$)
- **Final Complexity:** $O(n) + O(n) + O(n^2)$, which simplifies to $O(n^2)$.

D. Space Complexity: $O(n)$ "call stack"

exists

$$T(s) = \begin{cases} O(1) & \text{Search} < 0 \\ T(s-1) + 1 & \text{otherwise} \end{cases}$$
$$\begin{aligned} T(s) &= T(s-1) + 1 \\ &= T(s-2) + 1 + 1 \\ &= T(s-3) + 1 + 1 + 1 \\ &\vdots \\ &= T(s-k) + k \end{aligned}$$

assume $s-k = -1$ $k = s+1$

$$\begin{aligned} T(s) &= T(s-k) + k \\ &= T(-1) + s+1 \\ &= 1 + s+1 \\ &= O(s) = \boxed{O(n)} \end{aligned}$$

Count

$$T(i) = \begin{cases} T(i) \cdot O(1) & \text{index} = \text{size} \\ T(i+1) + O(i) & \text{otherwise} \end{cases}$$
$$\begin{aligned} T(i) &= T(i+1) + O(i) \\ &= T(i+2) + O(i) + O(i+1) \\ &= T(i+3) + O(i) + O(i+1) + O(i+2) \\ &\vdots \\ &= T(i+k) + O(i) + \dots + O(i+k-1) \end{aligned}$$

$i+k = \text{size}$ $k = \text{size} - i$

$$\begin{aligned} &T(n) + O(i) + \dots + O(s-1) \\ &= 1 + \sum_{j=1}^{s-i} O(i) \\ &= 1 + (O(1) + O(2) + \dots + O(s-1)) \\ &= 1 + \frac{n(n-1)}{2} \\ &= 1 + \frac{n^2 - n}{2} = \boxed{O(n^2)} \end{aligned}$$

4. Comparison:

Metric	Recursive	Non-Recursive
Best Case	$O(n)$	$O(n^2)$
Worst Case	$O(n^2)$	$O(n^2)$
Space Complexity	$O(n)$	$O(1)$